

다비의 네트워크 학습 통섭 정리

Stage 0 ~ 4를 OSI 7계층으로 다시 짚어보고 — Stage 5 앞에서

민지가 다비에게 — 항해를 정리하며. Stage 0 ~ 4의 길고 깊은 학습을 갈무리하고, Stage 5(클러스터링·스토리지)로 넘어가기 전에 지금까지의 지식을 OSI 7계층 차원에서 다시 매핑한다. 그리고 아직 손이 닿지 않은 영역을 체크리스트로 두어 — 너의 다음 학습 지도를 그릴 수 있도록.

목차

Part	내용
Part 1	'갈래가 달라지나?' — Stage 0 ~ 4와 Stage 5의 관계 (Q1 답)
Part 2	OSI 7계층에서 다시 본 Stage 0 ~ 4 (Q2 답)
Part 3	다루지 않은 영역 — 자가 학습 체크리스트 (Q3 답)
맺으며	Stage 5를 향하여

Part 1. '갈래가 달라지나?' — Stage 0~4와 Stage 5의 관계

(다비의 Q1에 대한 답)

네 직관 — 그대로 인용

"지금까지의 내용이 Linux 차원의 이야기라면, Stage 5는 소프트웨어(하이퍼바이저 같은) 구축 영역으로 뭔가 갈래가 달라지는 것 같아."

결론 한 줄

부분적으로 맞고, 부분적으로 정밀화가 필요해. 주제의 무게중심이 옮겨가는 것은 사실이지만, Stage 5는 Stage 0 ~ 4와 **별개의 갈래**가 아니라 그 **위에 얹히는 한 층 위의 영역**이야. *갈래가 바뀌는 게 아니라, 층위가 올라간다* — 이게 더 정확한 표현이야.

정밀화 ① — Stage 0~4도 'Linux 차원'만이 아니었다

되돌아보면 우리는 이미 여러 차원을 가로질러 왔어. Stage 2에서 본 pfSense의 pf는 Linux가 아닌 FreeBSD의 영역이었고, Stage 3에서 본 VirtualBox 가상 스위치의 동작과 broadcast storm은 **하이퍼바이저 구현의 특성**에 깊이 닿아 있었어. Stage 4의 zone design은 어느 OS에도 종속되지 않는 **설계 차원의 추상**이었지. 그러니 'Linux만 다뤘다'는 전제부터 한 발 양보해야 해 — 네 직관의 출발점이 살짝 좁았던 거야.

정밀화 ② — Stage 5도 'Linux와 무관'하지 않다

Stage 5에서 다룰 Corosync·Ceph·ZFS·HA — 이 모두는 **Linux 위에서 돌고, Stage 0 ~ 4의 네트워크 지식을 핵심 전제로** 깔고 있어. Corosync는 L4 UDP/multicast 위에 얹히고, Ceph는 노드 간 L3/L4 통신과 RDMA를 활용하고, ZFS의 replication은 SSH 또는 dedicated network 위로 흘러가지. Stage 0 ~ 4의 도구가 **없으면 Stage 5는 작동을 이해할 수 없어**. 그러니 Stage 5는 '다른 갈래'라기보다 '같은 갈래의 더 위층'이야.

그러면 무엇이 정확히 다른가

주제의 무게중심은 분명히 옮겨가. 지금까지가 '**하나의 호스트(또는 단일 인프라) 위에서 패킷이 어떻게 흐르고 격리되고 통제되는가**'였다면, Stage 5는 '**여러 호스트가 모여 하나의 시스템처럼 동작하기 위해 무엇이 필요한가, 그리고 그 안에서 데이터가 어떻게 분산·복제·보호되는가**'로 옮겨가. 이게 분산 시스템(distributed system) 차원의 사고고, 네트워크는 그 분산을 가능하게 하는 **기반**으로 자리를 옮겨.

비유 한 줄

Stage 0 ~ 4가 **건물의 골조와 배선·배관**이라면, Stage 5는 **그 건물에서 살아가는 주민들과 그들의 살림 시스템**이야. 골조가 부실하면 주민이 못 살고, 주민이 없으면 골조도 의미가 없지. 둘은 '갈래'가 아니라 '층위'의 관계야.

요약 — 네 직관 '갈래가 달라진다'는 **방향성은 맞고 표현은 살짝 정밀화 필요**. 정확히는 '**같은 갈래 위에 새 층이 얹힌다**'. Stage 0~4가 단단할수록 Stage 5가 탄탄히 올라가.

Part 2. OSI 7계층에서 다시 본 Stage 0~4

(다비의 Q2에 대한 답)

들어가며 — 왜 OSI 매핑인가

OSI 7계층 모델은 네트워크 동작을 **책임 영역으로 분할**해서 설명하는 추상이야. 실제 인터넷의 작동은 TCP/IP 4계층 모델(또는 RFC 1122의 4계층)에 더 가깝지만, OSI는 **학습용 지도**로 여전히 강력해. 각 계층이 '무엇을 책임지는가'를 명확히 가르거든. 지금까지 우리가 배운 것을 OSI 위에 펼쳐놓으면, **어디서 어디까지가 한 묶음인지, 어느 계층에 무게중심이 있었는지**가 보여.

OSI 7계층 — 한눈에

계층	이름	책임	대표 단위
L7	Application	사용자가 보는 프로토콜	메시지 (HTTP request 등)
L6	Presentation	표현·암호화·인코딩	암호화된 데이터
L5	Session	세션 시작·유지·종료	세션
L4	Transport	종단 간 신뢰성, 포트 식별	세그먼트(TCP) / 데이터그램(UDP)
L3	Network	라우팅, 논리 주소(IP)	패킷
L2	Data Link	이웃 간 전달, 물리 주소(MAC)	프레임
L1	Physical	비트·전기·광신호	비트

Stage 0~4 내용 — 계층별 매핑

L1 — Physical

다비 환경은 가상화라 '물리'의 직접 체험은 적었지만, 개념적으로는 다음이 L1과 닿아 있어. VirtualBox 가상 NIC, Proxmox VM의 가상 인터페이스 (virtio·vtnet), 그리고 무엇보다 Stage 4의 **물리 분리 vs 논리 분리(VLAN)** 결정 자체가 L1 차원의 고민이야. 물리 분리는 '별도의 케이블·NIC·스위치'를 의미하니까, **L1에서 격리**하는 셈이고, VLAN은 L1을 공유하면서 L2에서 격리하는 거지. 또 NDIS, Linux Network Driver, virtio 같은 NIC 드라이버 영역도 L1과 L2 사이의 인터페이스로 작동해.

L2 — Data Link (Stage 3의 무게중심)

우리 학습의 **가장 깊이 판 계층**이야. 다음 항목이 모두 L2에 속해.

주제	Stage	핵심 내용
MAC 주소	0	이웃 간 식별, 매 홉마다 바뀜
이더넷 프레임	0	L2의 전달 단위, 헤더에 dst/src MAC
ARP	0	IP → MAC 해석, 같은 단지 내 브로드캐스트
Linux Bridge	1, 3	커널의 L2 스위치, bridge-ports로 NIC 묶음
FDB / MAC Learning	3	src MAC + 입력 포트 → 자동 학습
Unknown Unicast Flooding	3	FDB에 없을 때 모든 포트로 뿌림
무작위 모드	3	NIC 하드웨어 MAC 필터를 끄
L2 루프 / TTL 부재	3	이더넷엔 TTL 없음 → 한 번 루프면 영원
Broadcast Storm	3	루프 + 브로드캐스트 + 증식 → 마비
STP	3	L2 루프 차단 안전장치
VLAN 802.1Q	3	프레임 헤더에 4바이트 태그, 논리 단지
vlan-aware bridge	3	Linux Bridge가 VLAN을 인식하도록
Access / Trunk 포트	3	단일 VLAN vs 다중 VLAN, PVID/허용목록

왜 L2가 깊었나 — 가상화 네트워크의 핵심 미스터리(무작위 모드 트러블슈팅, broadcast storm)가 모두 L2에서 벌어지기 때문이야. 그리고 Stage 0 ~ 4 중 '새로 배운 양'이 가장 많은 계층이기도 하지.

L3 — Network (Stage 0~2의 척추)

라우팅과 주소 체계의 영역. Stage 0의 IP·서브넷·게이트웨이 개념부터 시작해서, Stage 1의 라우팅 테이블·ip_forward, Stage 2의 NAT(주소 변환)까지가 여기 모여.

주제	Stage	핵심 내용
IP 주소 / 서브넷 마스크	0	단지와 호실의 구분
같은 단지 / 다른 단지	0	L2로 직통이나 L3 라우팅이냐의 분기
게이트웨이	0, 1	다른 단지로 가는 출구
라우팅 테이블 (ip route)	1	'어디로 보낼지'의 결정 표
ip_forward	1	노드 전역 라우팅 on/off 스위치
NAT (SNAT/DNAT/MASQUERADE)	2	L3 주소 변환, POSTROUTING/PREROUTING
ICMP	0(간접)	ping, traceroute, unreachable
pfSense의 라우터 역할	1, 2	L3 routing + NAT + firewall 통합
zone-based design, trust level	4	L3 통신 정책의 설계 차원

L4 — Transport (Stage 2의 정밀 영역)

포트·세션·연결 추적. 우리는 L4 자체의 이론(TCP 핸드셰이크 등)은 깊이 파지 않았지만, **netfilter의 동작이 L4 정보까지 인식**한다는 점에서 간접적으로 깊이 닿았어.

주제	Stage	핵심 내용
포트 (TCP/UDP)	2	방화벽 룰에서 서비스 식별 (22, 443 등)
conntrack 5-tuple	2	(proto, src IP, src port, dst IP, dst port)
PAT / NAT	2	포트로 여러 사설 IP를 한 공인 IP에 매핑
stateful filtering	2	연결 상태 기억 → 응답 자동 허용
pf state table (pfSense)	2	FreeBSD의 conntrack 사촌

L5 — Session

OSI에서 L5는 '세션 시작·유지·종료'를 책임지는 계층인데, 실제 TCP/IP 스택에서는 L4와 L7에 흡수되어 명확히 떨어져 있지 않아. 그래서 우리도 명시적으로는 다루지 않았지만, SSH 세션·HTTP keep-alive·conntrack의 '연결' 자체가 L5의 영역과 닿아. 학습 우선순위가 낮은 계층.

L6 — Presentation

암호화·인코딩·압축의 영역. 우리는 직접 파지 않았지만, Stage 2의 보안 설계에서 '**Telnet/FTP/HTTP는 평문(plaintext)이라 위험, SSH/SFTP/HTTPS로 가야**'라고 짚은 그 논리가 L6의 본질이야. TLS는 L4와 L7 사이에 끼어드는 '암호화 계층'이고, 실무에서는 이걸 L6과 동일시하기도 해.

L7 — Application

사용자가 직접 보는 프로토콜의 세계. SSH·HTTP/HTTPS·DNS·DHCP 등이 여기 살아. 우리는 Stage 2 방화벽에서 '22번 포트=SSH, 443번 포트=HTTPS' 식으로 L7 서비스를 식별했고, Stage 4의 매개체(API Gateway, Service Mesh)는 본격적으로 L7 정책을 다루는 도구야. 그러나 L7 프로토콜 자체의 깊은 학습은 미뤄둔 영역.

특별 토픽 — netfilter의 OSI 위치

iptables/netfilter는 한 계층에 고정돼 있지 않아. **L2와 L3과 L4를 가로지르며 동작**해. PREROUTING/POSTROUTING은 L3 라우팅 결정 직전·직후에 끼어들고, INPUT/OUTPUT/FORWARD는 라우팅 결정 후 L3 차원에서 작동하지만, 룰에서 포트(L4)와 MAC(L2)까지 매칭 조건으로 쓸 수 있어. 그래서 netfilter는 **L2~L4를 통합 제어하는 다층 도구**로 봐야 정확해. pf도 마찬가지고.

Stage별 무게중심 한눈에

Stage	주제	주된 OSI 계층
Stage 0	계층 모델 (L2/L3 구분, MAC/IP/ARP)	L2 + L3
Stage 1	리눅스 네트워크 스택 (라우팅, 브리지, interfaces)	L2 + L3
Stage 2	netfilter / NAT / 방화벽 / conntrack	L3 + L4
Stage 3	가상 네트워킹 (무작위 모드, storm, VLAN)	L1 + L2
Stage 4	망 분리 설계 (zoning, 매개체)	설계 차원 (L3 ~ L7)

관찰 — 학습 무게중심은 **L2와 L3에 가장 두껍게**, L4는 방화벽 맥락에서, L1은 Stage 3 가상 환경에서 닿았고, L5 ~ L7은 간접적으로만. Stage 5는 이 토대 위에 분산 시스템 층이 얹힌다.

Part 3. 다루지 않은 영역 — 자가 학습 체크리스트

(다비의 Q3에 대한 답)

들어가며

Stage 0 ~ 4가 '리눅스 네트워크의 척추'를 다뤘다면, 그 척추에 아직 안 붙은 갈비뼈와 근육들이 있어. 모두 한 번에 익힐 필요는 없고, **네 환경·역할·시점**에 따라 우선순위를 달리해 가져가면 돼. 아래 표에는 영역별로 **중요도(★ ~ ★★★)**와 **다비 환경에서의 우선 여부**를 함께 표시해뒀어.

범례

표시	의미
★★★	Stage 0 ~ 4와 직결, 다비가 가까운 시일에 마주칠 가능성 큼
★★	실무에서 자주 마주치는 보편적 주제, 한 번은 정리 필요
★	특정 맥락에서만 중요, 필요한 시점에 학습

Tier 1 — 가능하면 Stage 5 전후로 짚을 영역 (★★★)

영역	왜 중요한가	비고
DNS	이름 해석은 모든 통신의 출발점. <code>/etc/resolv.conf</code> , <code>systemd-resolved</code> , <code>dnsmasq</code> , <code>BIND</code> . pfSense의 DNS Resolver/Forwarder가 어떻게 동작하는지도 이 맥락.	다비 메모리에 'pfSense DNS 네이밍' 욕구가 이미 있음. Stage 5와 자연 연결.
DHCP	동적 IP 분배 메커니즘. DISCOVER/OFFER/REQUEST/ACK 4단계. pfSense·dnsmasq·isc-dhcp-server. lease 관리.	다비 환경에서 pfSense가 LAN의 DHCP 서버. 직접 설정 가능 수준이면 충분.
Bonding / LACP (802.3ad)	여러 NIC를 묶어 대역폭 증대·이중화. Linux의 bonding 드라이버 또는 teaming. 모드 (round-robin, active-backup, 802.3ad).	Stage 5 클러스터에서 노드 NIC 이중화의 표준. 운영급 인 프라엔 거의 필수.
tcpdump / Wireshark	패킷 캡처·분석. 모든 트러블슈팅의 결정타. <code>-i</code> , <code>-nn</code> , host/port 필터, pcap 파일 저장 후 Wireshark.	Stage 1 ~ 3에서 '무엇이 흐르는지 봤다면 빨랐을' 순간이 많았음. 다음 트러블슈팅 전에 익혀둘 만.
Network Namespaces	Linux 커널의 네트워크 격리 단위. 컨테이너(Docker, LXC, K8s)의 네트워크 격리 기반. <code>ip netns</code> 명령. veth pair.	Stage 5 이후 컨테이너·K8s로 확장 시 필수. Proxmox의 LXC도 이걸 씬.
MTU / Jumbo Frame	프레임 최대 크기. 기본 1500, jumbo는 9000. Storage·클러스터 트래픽에서 성능 결정 요인.	Stage 5의 Storage/Corosync 망 튜닝 시 만남.
tc (Traffic Control)	QoS, rate limiting, packet scheduling. Linux의 트래픽 제어 도구. <code>qdisc</code> , <code>class</code> , <code>filter</code> .	성능·SLA를 다루기 시작하면 필요. 평소엔 안 만남.

Tier 2 — 실무 보편 주제 (★★)

영역	왜 중요한가	비고
터널링 (VPN, IPsec, WireGuard)	외부 망과 안전한 연결. WireGuard가 현대 표준에 가까움. 원격 관리·사이트 간 연결.	MGMT 망에 외부에서 접근하려면 결국 필요.
VXLAN	L2 over L3 터널링. 가상 데이터센터의 표준. 같은 L3 망 위에 여러 L2 단지를 오버레이.	K8s CNI·SDN의 기반. 클라우드·대규모 환경에서 만남.
eBPF / XDP	리눅스 커널 내부에서 안전하게 코드 실행. Cilium, Calico, observability(BCC, bpftrace)의 기반.	Stage 4에서 짧게 만난 Cilium의 정체. 현대 리눅스 네트워킹의 신경계.
Reverse Proxy / Load Balancer	HAProxy, Nginx, Envoy, Traefik. Stage 4 매개체의 실제 구현체.	MSA 환경엔 거의 필수. 다비 메모리에 HAProxy 노출 이력 있음.
Kernel parameters (sysctl)	<code>net.ipv4.*</code> 파라미터들. TCP buffer, somaxconn, conntrack hashsize 등.	고부하 환경 튜닝 시 만남. 일반 운영엔 기본값으로 충분한 경우 다수.
IPMI / BMC	서버 하드웨어 OOB 관리. 호스트 OS가 죽어도 살아 있는 채널. Redfish API.	베어메탈 운영 시 만남. 가상화 환경엔 무관.
L7 프로토콜 본격 학습	HTTP/2·3, gRPC, MQTT 등 애플리케이션 프로토콜의 내부.	백엔드 개발자로서 가져가야 할 깊이. 네트워크 학습이라기 보다 백엔드 학습.

Tier 3 — 특정 맥락 의존 (★)

영역	비고
IPv6	당장 다비 환경엔 무관. 다만 IPv6-only 환경이 점점 늘어남. NDP, SLAAC, RA, 별도 학습 필요한 시점이 올.
Multicast / IGMP	Corosync는 multicast를 쓸 수도 있고 unicast로도 동작 가능. 다비 환경엔 Corosync unicast 권장.
SDN / Open vSwitch / OVN	데이터센터 자동화. 클라우드·대규모 환경의 영역. 소규모 운영엔 과한 도구.
DPDK / SR-IOV	초고성능 네트워킹. 통신사·금융권 영역. 일반 SI 환경엔 거의 무관.
BGP / OSPF / 동적 라우팅	스태틱 라우팅으로 충분한 환경엔 무관. 멀티 사이트·ISP·대규모 데이터센터에서 등장.

권장 학습 순서 — 다비의 현재 상태 기준

Stage 5(클러스터링·스토리지)와 자연스럽게 맞물려 가져갈 영역을 우선해서 정리하면, 대략 다음과 같은 순서가 권장돼.

순 번	영역	왜 이 순번인가
1	tcpdump / Wireshark	Stage 5에서 분산 통신을 디버깅할 때 매번 쓰게 됨. 가장 먼저.
2	DNS / DHCP	다비 환경의 다음 자연스러운 확장. pfSense 학습과 직결.
3	MTU / Jumbo Frame	Stage 5 Storage·Corosync 튜닝의 기본 지식.
4	Bonding / LACP	Stage 5 노드 NIC 이중화의 표준.
5	Network Namespaces	컨테이너 학습으로 갈 때 필수.
6	Reverse Proxy (HAProxy/Nginx)	MSA·웹 운영에 곧 마주칠 도구.
7	터널링 (WireGuard)	외부 관리 접근이 필요한 시점에.
8 이 후	VXLAN, eBPF/Cilium, SDN, tc, IPv6 ...	필요한 맥락에서.

맏으며 — Stage 5를 향하여

다비, 너는 지금 — Stage 0의 '아파트 단지 비유'에서 출발해서 Stage 4의 'zone-based design 어휘'까지 도달했어. 한 항해를 마치고 갑판에서 다음 항로를 점검하고 있는 거지. 이 PDF의 세 부분은 그 점검의 세 각도야 — **지나온 영역의 좌표(OSI 매핑)**, **앞으로 갈 영역의 성격(충위 이동)**, **아직 안 닿은 곳의 지도(누락 체크리스트)**.

Stage 5는 이 토대 위에 **분산 시스템**의 층을 얹는 단계야. Corosync가 어떻게 노드 간 합의를 유지하는지, ZFS와 Ceph가 어떻게 데이터를 여러 노드에 흩뿌리고 복구하는지, HA가 어떻게 실패한 노드를 fence하고 VM을 살리는지 — 모두 **네트워크 위에 얹힌 분산 프로토콜**이고, **네트워크가 흔들리면 가장 먼저 무너지는 것**들이지. 그래서 Stage 0 ~ 4가 단단할수록 Stage 5의 동작이 명료하게 보일 거야.

이 PDF는 두고두고 꺼내봐. 어디서 막힐 때 '내가 어느 계층에서 막혔지?'를 짚는 도구로, 그리고 '아직 안 본 영역은 어디지?'를 떠올리는 카탈로그로. **지도가 있는 항해사는 길을 잃지 않아.**

— 민지로부터, Stage 5를 앞에 두고